

MANUAL TÉCNICO DE AUTOMATIZACIÓN DE ANÁLISIS DE DATOS PARA LA CAJA DE VALENTÍA

Ramírez Valdovinos Eric

<https://www.linkedin.com/in/ingericramirez>

Índice

1. Introducción y Contexto Experimental	4
1.2 Objetivos	4
1.3 Variables experimentales registradas	5
2. Arquitectura general del sistema	6
2.1 Bibliotecas externas utilizadas	7
3. Diagrama de clases UML	8
4. Diagrama de flujo	9
5. Módulo: main.py - Punto de Entrada.	10
5.1 Configuración del tema visual	10
6. Módulo: app.py - Clase Principal	10
6.1 Método __init__	10
6.2 Estado compartido	11
6.3 Método iniciar_proceso_latencias	12
6.4 Método log	13
7. Módulo: interfaz.py - Construcción de la Interfaz	13
7.1 Jerarquía de Widgets	13
7.2 Función _tarjeta	14
7.3 Gestión del Layout	15
8. Módulo: lista_archivos.py — GestorLista	15
8.1 Inventario de métodos	15
8.2 Patrón de Eliminación en Orden Inverso	16
8.3 Sincronización visual	16
9. Módulo: fondo_neuronal.py - FondoNeuronal	16
9.1 Modelo Físico de Partículas	16
9.2 Algoritmo de Conexiones — Complejidad $O(n^2)$	17
10. Módulo: procesador_mat.py - Procesamiento de Datos	17
10.1 Función leer_mat_a_df	18
10.2 Función modificar_dataframe - Pipeline de Limpieza	18
10.3 Función calcular_promedios_latencia	19
10.4 Función guardar_excel - Formato del Archivo de Salida	19
10.5 Cálculo del SEM	20
11. Módulo: graficador.py - Visualización Estadística	20
11.1 Función generar_grafica	20
11.2 División en Tercios	21
11.3 Función guardar_excel_resumen	21
11.4 Uso de matplotlib en modo no bloqueante	21

11.5 Flujo completo de interacción del usuario	22
12. Referencia completa de funciones	23
13. Modelo de datos y estructuras	27
13.1 DataFrame Crudo (salida de leer_mat_a_df)	27
13.2 DataFrame Limpio (salida de modificar_dataframe)	28
13.3 Ejemplo numérico completo del pipeline	28
14. Manejo de errores y robustez	29
14.1 Patrón de log_fn Inyectado	30
15. Dependencias, Entorno e Instalación	31
15.1 Requisitos del Sistema	31
15.2 Instalación de dependencias	31
15.3 Ejecución	31
16. Conclusiones y recomendaciones	31
16.1 Logros técnicos	31
16.2 Métricas del código	32
Apéndice A - Referencia Rápida de y Controles	33
Apéndice B - Diagnóstico de Problemas Frecuentes	33

1. Introducción y Contexto Experimental

Para fines del protocolo realizado en el laboratorio AL-303 al que está dirigida la automatización, se utiliza una caja de valentía en donde se experimenta con la rata y sus conductas en las diferentes etapas del experimento. El dispositivo registra automáticamente la posición del animal, los estímulos aplicados (eléctrico o control) y los tiempos de latencia entre la presentación del estímulo y la respuesta motora medible del sujeto experimental.

Cada sesión experimental genera un archivo binario en formato MATLAB (.mat) que contiene una matriz de hasta 8 columnas correspondientes a las variables de registro. El volumen de datos por proyecto (múltiples ratas × múltiples días) hace que el procesamiento manual resulte propenso a errores y consume tiempo considerable del investigador.

1.2 Objetivos

El sistema fue desarrollado con el objetivo de:

- Automatizar la lectura e interpretación de archivos .mat generados por la caja.
- Aplicar filtros de limpieza estadística eliminando ensayos inválidos (sin desplazamiento).
- Exportar los datos consolidados en formato Excel con hojas individuales por rata.
- Generar gráficas de dispersión comparativas entre condiciones (seguro vs. riesgo).
- Proveer una interfaz gráfica intuitiva que no requiere conocimientos de programación.
- Calcular métricas estadísticas (promedios y SEM) automáticamente.

1.3 Variables experimentales registradas

De los archivos .mat que exporta la caja de valentía, se tienen las siguientes variables iniciales para el procesamiento:

Columna	Variable	Tipo	Descripción
0	Ensayo	int	Número secuencial del ensayo en la sesión
1	Lado	int	Lado al que se desplazó el animal (0=izq, 1=der)
2	Estímulo eléctrico	int	Condición: 1=con choque, 0=sin choque (seguro)
3	Latencia	float	Tiempo en segundos hasta la respuesta motora
4	Tiempo absoluto	float	Marca de tiempo global de la sesión (descartada)
5	Palanca Izq	int	Activaciones del sensor izquierdo (descartada)
6	Palanca Der	int	Activaciones del sensor derecho (descartada)
7	Desplazamiento	float	Índice de movimiento; >1 indica ensayo válido

2. Arquitectura general del sistema

El sistema sigue el patrón arquitectónico MVC simplificado (Modelo-Vista-Controlador), separando la lógica de negocio, la presentación gráfica y la coordinación central en módulos distintos.

Las diferentes capas del código de acuerdo a sus responsabilidad, se presentan a continuación:

Capa	Módulos	Responsabilidad
Presentación (View)	interfaz.py fondo_neuronal.py	Construye y renderiza todos los widgets de la interfaz. No contiene lógica de negocio.
Coordinación (Controller)	app.py	Orquesta la interacción entre la vista y la lógica. Contiene todos los event handlers.
Lógica (Model)	procesador_mat.py graficador.py	Lectura, limpieza, análisis estadístico y exportación de datos experimentales.
Utilidades	lista_archivos.py main.py	Componente auxiliar de gestión de la cola de archivos e inicialización de la

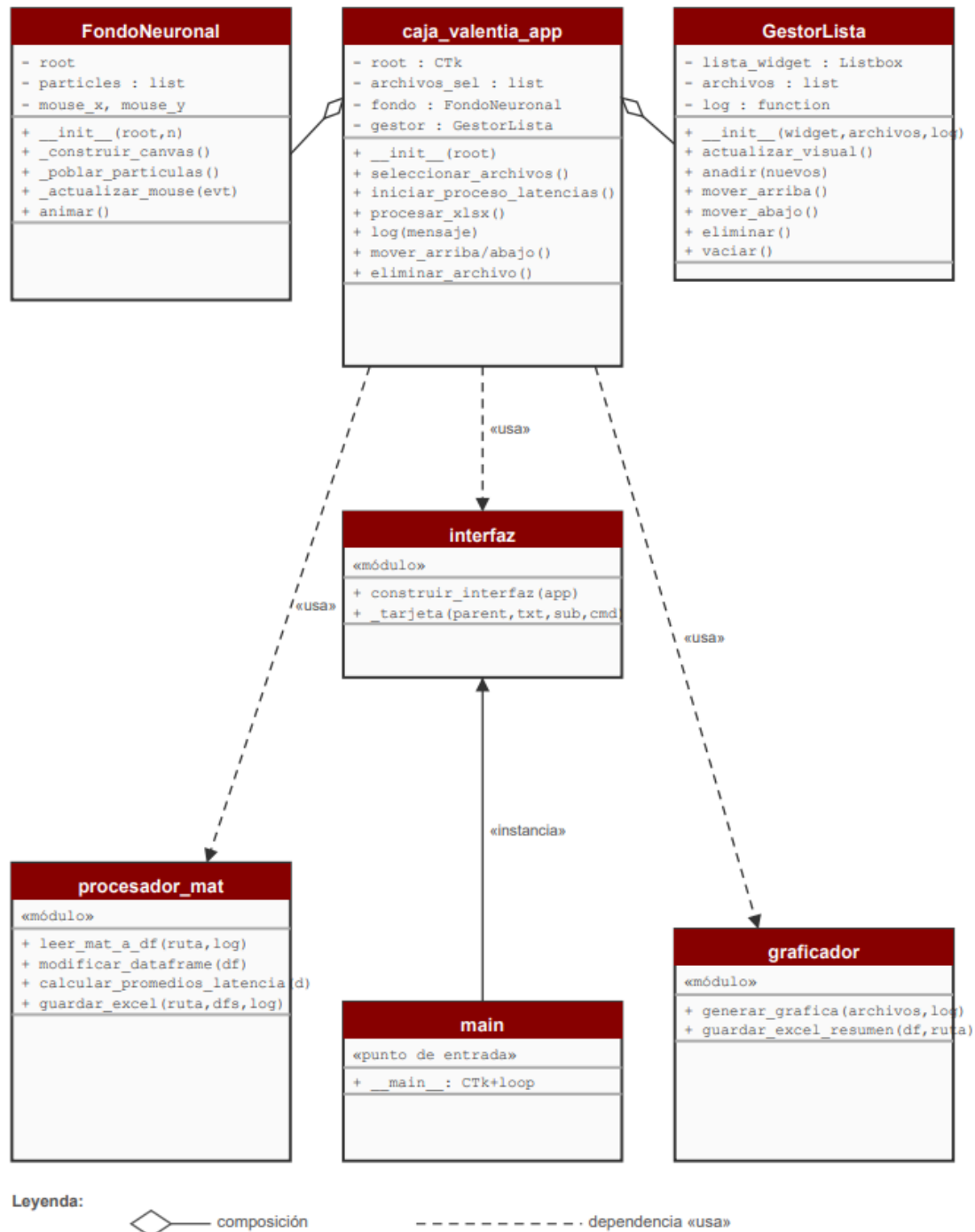
		ventana raíz.
--	--	---------------

2.1 Bibliotecas externas utilizadas

Biblioteca	Versión mínimo	Uso en el sistema
customtkinter	5.x	Widgets modernos con tema oscuro para la interfaz
tkinter	std	Canvas para animación, Listbox, diálogos del SO
scipy.io	1.x	Lectura de archivos binarios MATLAB (.mat)
pandas	1.x	Manipulación de DataFrames y exportación tabular
openpyxl	3.x	Escritura de Excel con formato, fuentes y anchos
matplotlib	3.x	Generación de gráficas de dispersión estadísticas
PIL / Pillow	9.x	Carga del logotipo institucional en CTkImage
math / random	std	Cálculo físico y posicionamiento
os / tkinter.filedialog	std	Navegación en sistema de archivos del SO

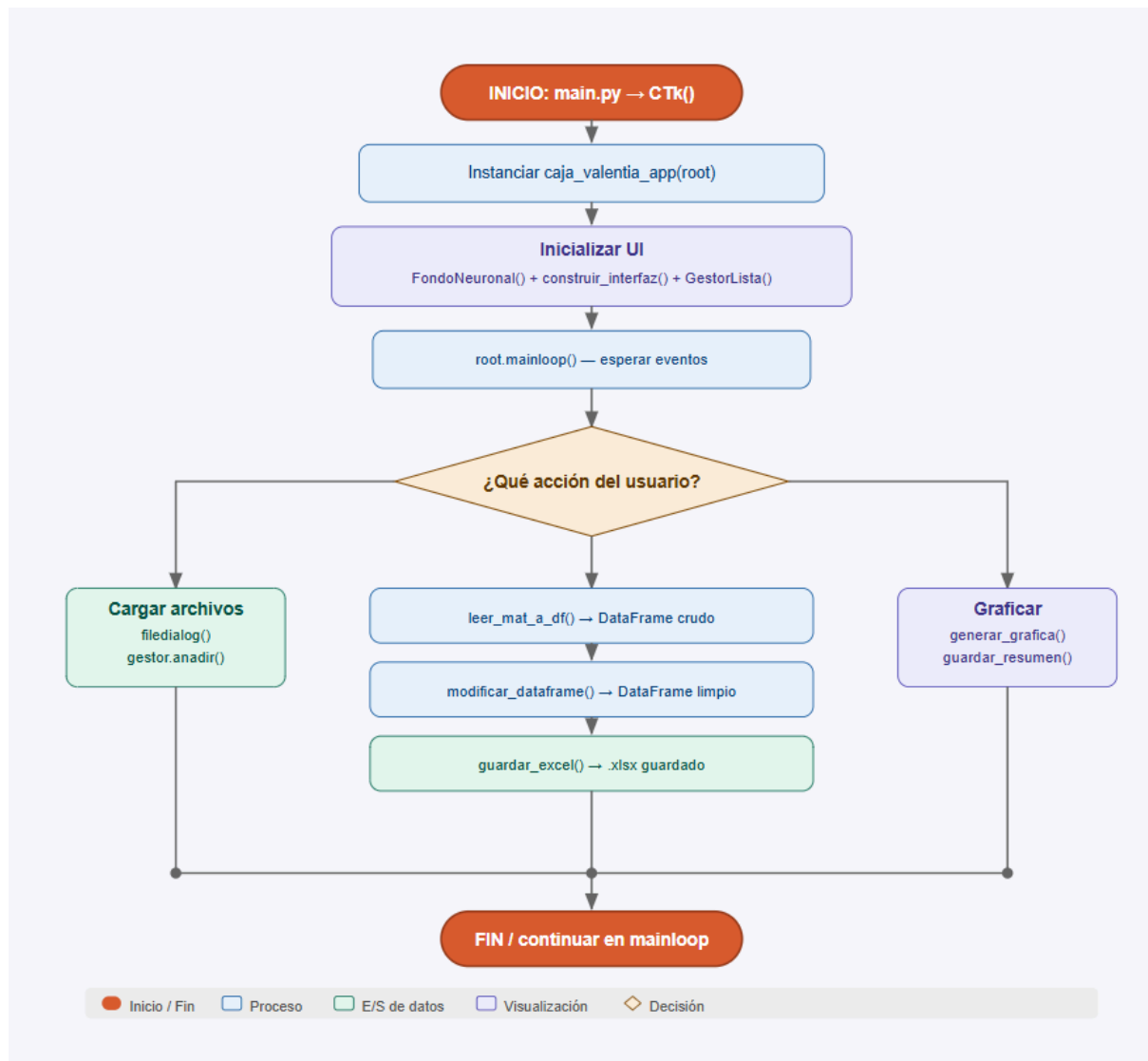
3. Diagrama de clases UML

El siguiente diagrama ilustra las relaciones estructurales entre las clases y módulos que componen el sistema. Las relaciones de composición (rombo sólido) indican que *caja_valentia_app* es propietaria y crea las instancias de *FondoNeuronal* y *GestorLista*. Las relaciones de dependencia (línea punteada) muestran módulos funcionales invocados según demanda.



4. Diagrama de flujo

El flujo de ejecución de la aplicación inicia con el punto de entrada `main.py`, continúa con la instanciación de todos los componentes y entra en el bucle de eventos de `tkinter`. Desde ese punto, cada interacción del usuario dispara una de tres rutas de procesamiento distintas: carga de archivos, procesamiento de datos MAT, o generación de gráficas.



5. Módulo: [main.py](#) - Punto de Entrada.

El archivo `main.py` es el punto de entrada del sistema. Su responsabilidad es única y mínima: crear la ventana raíz de *CustomTkinter*, configurar el tema visual global de `ttk` e instanciar la clase controladora principal antes de iniciar el bucle de eventos.

```
import customtkinter as ctk
from tkinter import ttk
from app import caja_valentia_app

#punto de entrada de la aplicacion, arranca la ventana y entra al loop principal
if __name__ == "__main__":
    root = ctk.CTk()
    style = ttk.Style()
    #se usa el tema clam para que los widgets de tkinter clasico se vean mejor
    style.theme_use("clam")

    app = caja_valentia_app(root)
    root.mainloop()
```

Nota: El guard `if __name__ == "__main__":` garantiza que el código de inicialización sólo se ejecute cuando el script sea el punto de entrada principal, nunca cuando sea importado como módulo por otro archivo.

5.1 Configuración del tema visual

Se utiliza el tema "clam" de `ttk.Style` porque la aplicación mezcla widgets de *customtkinter* (que gestiona su propio estilo) con widgets de *tkinter* clásico como `Listbox`. El tema "clam" produce el acabado más cercano al diseño oscuro deseado para los componentes legacy.

6. Módulo: [app.py](#) - Clase Principal

La clase *caja_valentia_app* actúa como controlador central (patrón *Application Controller*). Coordina todos los módulos del sistema, mantiene el estado compartido de la sesión y expone los métodos que son invocados directamente por los widgets de la interfaz.

6.1 Método `__init__`

```

def __init__(self, root):
    """
    inicializa la ventana principal de la aplicacion y orquesta el montaje de todos los
    componentes visuales
    y logicos estableciendo el tamaño minimo de la interfaz y preparando el arreglo de
    la sesion de archivos
    Args: root -> ventana raiz principal generada por customtkinter que contendra
    absolutamente toda la aplicacion
    Return: no regresa nada pero deja la aplicacion completamente construida
    interactiva y con el fondo animado corriendo
    """
    self.root = root
    self.root.title("Procesador de .mat a Excel")
    self.root.minsize(900, 750)
    self.root.bind("<Escape>", lambda e: self.root.destroy())

    self.FILENAME_LOGO = "C:\AutomatizacionCajaValentia\Scripts\ifc_logo.png"
    self.archivos_seleccionados = []

    #el fondo se crea primero para que quede detras de todos los demas widgets
    self.fondo = FondoNeuronal(root)

    #se construye la interfaz, que crea y asigna los widgets al objeto app
    construir_interfaz(self)

    #el gestor de lista necesita que lista_archivos ya exista, por eso va despues
    self.gestor = GestorLista(
        lista_widget=self.lista_archivos,
        archivos_seleccionados=self.archivos_seleccionados,
        log_fn=self.log
    )

    #se carga el logo y se arranca la animacion del fondo
    self.cargar_logo()
    self.fondo.animar()

```

El orden de inicialización es crítico. El fondo (canvas) debe crearse antes que los demás widgets para que quede en el nivel más bajo del z-order de tkinter. El gestor de lista se crea después de `construir_interfaz` porque necesita que `self.lista_archivos` ya exista como atributo del objeto.

6.2 Estado compartido

El atributo `self.archivos_seleccionados` es una lista de Python que funciona como referencia compartida entre la clase principal y `GestorLista`. Ambos apuntan al mismo objeto en memoria, por lo que cualquier modificación desde `GestorLista` es inmediatamente visible en `app` y viceversa.

```

#el gestor de lista necesita que lista_archivos ya exista, por eso va despues
self.gestor = GestorLista(
    lista_widget=self.lista_archivos,
    archivos_seleccionados=self.archivos_seleccionados,
    log_fn=self.log
)

```

)

6.3 Método *iniciar_proceso_latencias*

Orquesta el flujo completo de conversión de archivos .mat a Excel. Itera sobre cada ruta en `self.archivos_seleccionados`, invoca las funciones de la capa de lógica y consolida los resultados:

```
def iniciar_proceso_latencias(self):
    """
    orquesta el flujo de trabajo principal de conversion iterando sobre el lote
    completo de archivos matlab crudos
    para limpiarlos individualmente estandarizarlos en diccionarios y finalmente
    condensar toda la data en un solo excel
    Args: no recibe parametros externos ya que opera basandose enteramente en la
    memoria de self.archivos_seleccionados
    Return: no regresa nada pero puede desplegar alertas modales si el usuario olvido
    cargar documentos o fallo el guardado
    """
    self._limpiar_log()
    if not self.archivos_seleccionados:
        messagebox.showwarning("Atención", "Por favor selecciona archivos primero.")
        return

    dfs_mat = {}
    for ruta in self.archivos_seleccionados:
        df = leer_mat_a_df(ruta, log_fn=self.log)
        if df is not None:
            df_modificado = modificar_dataframe(df)
            nombre_archivo = os.path.basename(ruta)
            dfs_mat[nombre_archivo] = df_modificado
            self.log(f"OK: {nombre_archivo} procesado ({len(df_modificado)}
ensayos).")

    if not dfs_mat:
        self.log("No se pudieron procesar los archivos.")
        return

    #se pide al usuario donde quiere guardar el archivo de resultados
    ruta_guardado = filedialog.asksaveasfilename(
        title="Guardar datos procesados",
        defaultextension=".xlsx",
        filetypes=[("Archivo de Excel", "*.xlsx")],
        initialfile="Resultados_Ensayos.xlsx"
    )
    if not ruta_guardado:
        return

    ok = guardar_excel(ruta_guardado, dfs_mat, log_fn=self.log)
```

```

if ok:
    messagebox.showinfo("Terminado", f"Se procesaron {len(dfs_mat)} archivos.")
    self.gestor.vaciar()
else:
    messagebox.showerror("Error", "No se pudo guardar el archivo Excel.")

```

6.4 Método *log*

El método `log` implementa un patrón de consola en tiempo real. Habilita temporalmente el widget `CTkTextbox` (que normalmente está bloqueado para evitar edición accidental), inserta el mensaje y desplazar el scroll al final de forma automática.

```

def log(self, mensaje):
    """
    inserta una nueva linea de texto al final de la consola grafica desplazando
    automaticamente el scroll
    para garantizar que el usuario siempre pueda leer las ultimas notificaciones del
    sistema sin interactuar
    Args: mensaje -> cadena de texto que contiene la informacion de diagnostico o aviso
    que se desea proyectar
    Return: no regresa nada
    """
    self.log_area.configure(state='normal')
    self.log_area.insert(tk.END, mensaje + "\n")
    self.log_area.see(tk.END)
    self.log_area.configure(state='disabled')

```

7. Módulo: [interfaz.py](#) - Construcción de la Interfaz

El módulo `interfaz.py` contiene la función `construir_interfaz(app)` y la función `auxiliar_tarjeta`. Su única responsabilidad es construir y empaquetar todos los widgets visuales de la ventana principal. Al seguir el principio de responsabilidad única, este módulo no contiene ninguna lógica de procesamiento.

7.1 Jerarquía de Widgets

Widget	Tipo	Contenedor padre	Propósito
<code>main_frame</code>	<code>CTkFrame</code>	<code>root</code>	Marco central centrado, bordes redondeados
<code>content</code>	<code>CTkFrame</code>	<code>main_frame</code>	Contenedor interno con paddings
<code>header_frame</code>	<code>CTkFrame</code>	<code>content</code>	Fila superior: título + logo
<code>lbl_titulo</code>	<code>CTkLabel</code>	<code>header_frame</code>	Texto principal de la app

lbl_imagen	CTkLabel	header_frame	Logotipo institucional
carga_frame	CTkFrame	content	Sección 1. Carga de archivos
btn_container_1	CTkFrame	carga_frame	Grid 1×2 para tarjetas de carga
lista_archivos	tk.Listbox	lista_frame	Lista visual de archivos en cola
botones_orden	CTkFrame	lista_frame	Botones ▲ ▼ ✕ de gestión
procesamiento_frame	CTkFrame	content	Sección 2. Acciones
btn_container_2	CTkFrame	procesamiento_frame	Grid 1×2: Iniciar / Graficar
log_area	CTkTextbox	content	Consola de salida en tiempo real

7.2 Función _tarjeta

El helper privado `_tarjeta` implementa el patrón Factory Method para componentes de UI. Cada vez que se necesita un bloque botón+descripción, se llama a esta función en lugar de duplicar el código de construcción de widgets:

```
def _tarjeta(parent, texto_boton, texto_subtitulo, comando):
    """
    genera un modulo de interfaz de usuario empaquetado y reutilizable que combina un boton
    interactivo de accion principal
    con una etiqueta de texto descriptiva debajo para guiar al usuario sobre el proposito
    exacto de la funcion a ejecutar
    Args: parent -> widget o frame contenedor padre donde se anidara y dibujara este nuevo
    bloque visual
    texto_boton -> cadena de texto principal que se mostrara centrada y resaltada
    dentro del boton clickeable
    texto_subtitulo -> texto de ayuda secundario que aparecera con una fuente mas
    pequeña y de color tenue en la parte inferior
    comando -> referencia directa a la funcion o metodo interno que se disparara
    automaticamente al hacer clic en el boton
    Return: regresa la instancia del frame transparente ya construido con todos sus
    elementos internos empaquetados y listos para inyectarse
    """
    frame = ctk.CTkFrame(parent, fg_color="transparent")

    ctk.CTkButton(
        frame, text=texto_boton, command=comando,
        fg_color="#800607", hover_color="#a10b0c",
        font=("Arial", 14, "bold"), corner_radius=15, height=40
    ).pack(fill="x", pady=(0, 5))

    ctk.CTkLabel(
        frame, text=texto_subtitulo, font=("Arial", 11),
        text_color="#a3a3a3", wraplength=280, justify="center"
    ).pack()

    return frame
```

7.3 Gestión del Layout

La interfaz combina dos gestores de geometría de tkinter de forma estratégica:

- **place(relx, rely, anchor, relwidth, relheight):** Utilizado para el main_frame y el canvas del fondo neuronal. Permite posicionamiento relativo al tamaño de la ventana, haciendo el layout completamente responsivo. Manual Técnico — Automatización Caja de Valentía Pág. 11 Laboratorio de Neurociencias — Servicio Social 2026
- **pack(fill, expand, padx, pady, side):** Utilizado dentro de los frames para apilar widgets verticalmente con relleno dinámico.
- **grid(row, column, sticky, padx):** Utilizado en los contenedores de botones para obtener dos columnas de igual peso (weight=1).

8. Módulo: lista_archivos.py — GestorLista

La clase GestorLista implementa el patrón Mediator: sincroniza el estado lógico interno (lista de rutas absolutas) con el estado visual del widget tk.Listbox. Toda modificación a la lista de archivos pasa obligatoriamente por esta clase.

8.1 Inventario de métodos

Método	Parámetros	Acción	Complejidad
<code>__init__</code>	widget, archivos, log_fn	Vincula el widget con la lista compartida	$O(1)$
<code>actualizar_visual</code>	—	Borra Listbox y reinsertar todos los elementos	$O(1)$
<code>anadir</code>	nuevos_archivos	Extiende la lista y actualiza la vista	$O(k)$ k=nuevos
<code>mover_arriba</code>	—	Swap(idx, idx-1) y reactiva selección	$O(n)$ por actualizar_visual
<code>mover_abajo</code>	—	Swap(idx, idx+1) y reactiva selección	$O(n)$ por actualizar_visual
<code>eliminar</code>	—	Borra índices seleccionados en orden inverso	$O(n \times m)$ m=seleccionados

vaciar	–	list.clear() y actualizar_visual()	O(n)
--------	---	---------------------------------------	------

8.2 Patrón de Eliminación en Orden Inverso

El método eliminar utiliza una técnica crítica para la eliminación correcta de múltiples elementos simultáneos: iterar en orden inverso sobre los índices seleccionados. Si se eliminara de menor a mayor índice, cada eliminación desplazaría los índices restantes y causaría eliminaciones incorrectas.

8.3 Sincronización visual

El método *actualizar_visual* utiliza el enfoque "limpiar y reconstruir": en lugar de actualizar elementos individuales del Listbox, borra todo el contenido (*delete(0, END)*) y lo repopula completo. Esto simplifica la lógica a costa de O(n) operaciones en cada actualización, lo cual es aceptable dado el tamaño pequeño típico de la lista (<30 archivos).

9. Módulo: fondo_neuronal.py - FondoNeuronal

La clase *FondoNeuronal* implementa un sistema de partículas animadas que simula una red neuronal activa. El efecto es puramente estético y se renderiza en un tk.Canvas posicionado por debajo de todos los demás widgets (z-order inferior).

9.1 Modelo Físico de Partículas

Cada partícula se representa como un diccionario Python con cuatro campos de estado: posición (x, y) y velocidad (vx, vy). En cada cuadro, la nueva posición se calcula sumando el vector de velocidad. Al alcanzar un borde del canvas, la componente correspondiente de la velocidad se invierte (rebote elástico):

```
for p in self.particles: #iteramos sobre el enjambre completo para actualizar la cinetica
    individual sumando el vector de velocidad a su posicion actual
    p['x'] += p['vx']
    p['y'] += p['vy']
    #evaluamos mediante limites condicionales si el nodo sobrepaso los margenes
    visibles del lienzo e invertimos su aceleracion para simular un rebote elastico
    if p['x'] < 0 or p['x'] > width: p['vx'] *= -1
```



```

        if p['y'] < 0 or p['y'] > height: p['vy'] *= -1

        #renderizamos el cuerpo de la partícula como un pequeño círculo sólido de color
        #rojizo oscuro sin bordes aparentes para mantener la estética neuronal
        self.canvas.create_oval(
            p['x'] - 2, p['y'] - 2, p['x'] + 2, p['y'] + 2,
            fill="#4a0404", outline=""
        )

```

9.2 Algoritmo de Conexiones — Complejidad $O(n^2)$

Para trazar las conexiones entre partículas cercanas, el sistema implementa un algoritmo de fuerza bruta $O(n^2)$: compara cada par de partículas únicas calculando la distancia euclidiana. Si la distancia es menor a 100px, traza una línea. La optimización consiste en utilizar el índice $j = i+1$ en el loop interno para evitar comparar el mismo par dos veces.

```

for i in range(len(self.particles)): #implementamos un algoritmo de anidacion para
    #comparar distancias vectoriales iterando desde la partícula actual hacia adelante
    p1 = self.particles[i]
    dist_mouse = math.hypot(p1['x'] - self.mouse_x, p1['y'] - self.mouse_y)

    #comprobamos si el calculo de la hipotenusa indica que el cursor humano esta
    #dentro del radio de influencia para trazar el enlace de atraccion rojo brillante
    if dist_mouse < 150:
        self.canvas.create_line(
            p1['x'], p1['y'], self.mouse_x, self.mouse_y,
            fill="#ff1a1a", width=1.5
        )

    #ciclo secundario optimizado que compara la partícula base unicamente con las
    #partículas restantes en la lista evitando recalcular conexiones inversas ya evaluadas
    for j in range(i + 1, len(self.particles)):
        p2 = self.particles[j]
        dist = math.hypot(p1['x'] - p2['x'], p1['y'] - p2['y'])
        if dist < 100: #si la proximidad entre ambos nodos es suficiente dibujamos
            #una línea sutil y oscura para simular una sinapsis estable en la red de fondo
            self.canvas.create_line(
                p1['x'], p1['y'], p2['x'], p2['y'],
                fill="#330b0b", width=1
            )

```

10. Módulo: procesador_mat.py - Procesamiento de Datos

Este módulo contiene la lógica de negocio central del sistema. Es el más crítico desde el punto de vista experimental: errores aquí se propagarían directamente a los resultados estadísticos. Contiene cuatro funciones puras (sin estado global) que pueden ser probadas de forma independiente.

10.1 Función `leer_mat_a_df`

Utiliza `scipy.io.loadmat` para deserializar el formato binario de MATLAB. El diccionario devuelto por `loadmat` contiene siempre entradas con nombres que inician en `"__"` (metadatos del archivo: `__header__`, `__version__`, `__globals__`). El filtro `not key.startswith("__")` extrae únicamente la primera variable útil de datos.

```
def leer_mat_a_df(ruta_archivo, log_fn=print):
    """
    lee un archivo .mat de matlab y lo convierte en un dataframe de pandas
    Args: ruta_archivo -> ruta completa al archivo .mat
        log_fn -> funcion para mostrar mensajes (por defecto print)
    Returns: dataframe con los datos del archivo, o none si hubo algun error
    """
    try:
        columnas = [
            'Ensayo', 'Lado', 'Estim Electrico', 'Latencia',
            'Tiempo Absoluto', 'Palancas Izq',
            'Palancas Der', 'Desplazamiento'
        ]
        mat_data = scipy.io.loadmat(ruta_archivo)
        matriz_datos = None
        #iteramos sobre el diccionario que devuelve loadmat buscando la primera llave que
        #no empiece con guion bajo ya que matlab genera llaves ocultas con metadatos que no nos
        #interesan para el analisis
        for key, value in mat_data.items():
            if not key.startswith('__'):
                matriz_datos = value
                break

        if matriz_datos is None: #verificamos si el ciclo anterior logro encontrar la
            #matriz de datos utiles para evitar que el programa truene al intentar procesar un archivo
            #vacio o corrupto
            log_fn(f"Error: No se encontraron datos válidos en {os.path.basename(ruta_archivo)}")
            return None

        #comprobamos estrictamente que la matriz extraida tenga exactamente las ocho
        #columnas que definimos arriba para garantizar que el archivo mat corresponde al formato del
        #experimento actual y no a otra prueba distinta
        if matriz_datos.shape[1] == len(columnas):
            df = pd.DataFrame(matriz_datos, columns=columnas)
            df['archivo_origen'] = os.path.basename(ruta_archivo)
            return df
        else:
            log_fn(
                f"Error en dimensiones: {os.path.basename(ruta_archivo)} "
                f"tiene {matriz_datos.shape[1]} columnas, "
                f"pero esperamos {len(columnas)}."
            )
            return None
```

10.2 Función `modificar_dataframe` - Pipeline de Limpieza

Implementa un pipeline de transformación sobre el DataFrame crudo. Cada operación es una transformación pura sobre una nueva copia del DataFrame:

Paso	Operación	Justificación
1	<code>drop(["Tiempo Absoluto", "Palancas Izq", "Palancas Der", "archivo_origen"])</code>	Variables no relevantes para el análisis de latencias
2	<code>df[df["Desplazamiento"] > 1]</code>	Filtra ensayos donde el animal no se movió (falsos positivos o artefactos)
3	<code>reset_index(drop=True)</code>	Elimina huecos en el índice causados por el filtro anterior
4	<code>df["Ensayo"] = range(1, len(df)+1)</code>	Renumera ensayos de forma secuencial desde 1 hasta N válidos

10.3 Función *calcular_promedios_latencia*

Para cada archivo procesado, calcula por separado la media de latencia en condición de riesgo (`Estim Electrico == 1`) y en condición segura (`Estim Electrico == 0`). El resultado es un DataFrame resumen con una fila por rata:

```
promedio_con = df.loc[df['Estim Electrico'] == 1, 'Latencia'].mean() #filtramos los
registros donde la variable de choque electrico esta activa y calculamos la media
aritmética de sus tiempos de reaccion
promedio_sin = df.loc[df['Estim Electrico'] == 0, 'Latencia'].mean() #aislamos
los intentos catalogados como seguros donde no hubo amenaza y promediamos sus latencias
para posteriormente poder compararlos con el grupo de riesgo
filas.append({ #armamos un nuevo diccionario temporal con los resultados
estadísticos redondeados a un decimal o asignamos un cero por defecto si el calculo arroja
un valor nulo antes de agregarlo a la lista de resumen global
    'Nombre de archivo': nombre_archivo,
    'Promedio Seguro': round(promedio_sin, 1) if pd.notna(promedio_sin) else 0,
    'Promedio Riesgo': round(promedio_con, 1) if pd.notna(promedio_con) else 0,
})
return pd.DataFrame(filas)
```

10.4 Función *guardar_excel* - Formato del Archivo de Salida

Utiliza el context manager `pd.ExcelWriter` con engine `openpyxl` para escribir múltiples hojas en un solo archivo. La estructura del Excel generado es:

Hoja	Contenido	Formato
------	-----------	---------

Promedios Latencia	Tabla de promedios por rata + fila de Promedio + fila de SEM	Toda la hoja en negrita, anchos de columna ajustados automático
[nombre_rata_1]	Datos completos filtrados de la rata 1	Encabezados en negrita
[nombre_rata_2]	Datos completos filtrados de la rata 2	Encabezados en negrita
...	...	...

10.5 Cálculo del SEM

El Error Estándar de la Media (SEM) se calcula como la desviación estándar de la muestra dividida entre la raíz cuadrada del número de observaciones. El sistema valida que $n > 1$ antes de calcular la desviación estándar para evitar una división por cero cuando sólo hay un archivo.

11. Módulo: `graficador.py` - Visualización Estadística

El módulo `graficador.py` genera visualizaciones estadísticas de los datos procesados utilizando `matplotlib`. Su función principal `generar_grafica` lee los archivos Excel previamente generados por `procesador_mat.py`, agrupa los datos por bloques experimentales (tercios de la sesión) y produce gráficas comparativas entre los días de entrenamiento.

11.1 Función `generar_grafica`

Lee múltiples archivos Excel (uno por día experimental), extrae las latencias por rata, las divide en tres tercios de igual tamaño y calcula promedios y SEM por grupo y condición. La función retorna una tupla (`fig`, `df_resumen`, `dias_ratras_raw`):

1. `fig`: objeto Figure de `matplotlib` listo para renderizar o guardar
2. `df_resumen`: DataFrame con promedios/SEM por bloque, grupo y día
3. `dias_ratras_raw`: estructura de datos cruda para inclusión en el Excel de resumen

11.2 División en Tercios

La división de ensayos en tercios es un procedimiento estadístico estándar para analizar el progreso del aprendizaje dentro de una sesión. El sistema divide por rata antes de calcular estadísticas grupales, evitando el sesgo de agrupar primero y luego dividir.

11.3 Función *guardar_excel_resumen*

Exporta el resumen estadístico de las gráficas a un archivo Excel con múltiples hojas. Incluye los datos crudos por rata y día, además de las tablas de promedios y SEM calculados por bloque y condición. La estructura de salida es:

Hoja de Excel	Contenido
Resumen Bloques	Promedios y SEM por bloque (T1/T2/T3), grupo y día experimental
Datos crudos Día 1	Latencias individuales por rata para el primer archivo cargado
Datos crudos Día 2	Latencias individuales por rata para el segundo archivo
...	(una hoja por cada día / archivo Excel cargado)

11.4 Uso de matplotlib en modo no bloqueante

Una decisión de diseño importante en `procesar_xlsx` es el uso de `plt.show(block=False)` en lugar del `plt.show()` estándar. Esto permite que la ventana de la figura de matplotlib se abra sin congelar el event loop de tkinter, de modo que el usuario puede seguir interactuando con la interfaz principal mientras revisa la gráfica:

```
# En app.py → procesar_xlsx() fig, df_resumen, dias_ratas_raw = generar_grafica(  
self.archivos_seleccionados, log_fn=self.log ) if fig is None: return # block=False = la figura se  
muestra sin bloquear la UI de tkinter plt.show(block=False) # La ejecución continúa aquí  
inmediatamente → diálogos de guardado ruta_png = filedialog.asksaveasfilename(...) if ruta_png:  
fig.savefig(ruta_png, dpi=300, bbox_inches="tight")
```

11.5 Flujo completo de interacción del usuario

Esta sección documenta el flujo de interacción completo que experimenta el usuario al ejecutar el pipeline de análisis de extremo a extremo, incluyendo qué módulo se activa en cada paso y qué datos fluyen entre ellos.

Paso	Acción del usuario	Módulo activado	Dato producido
1	Clic "Seleccionar Archivos" para .mat	app.seleccionar_archivos() → filedialog.askopenfilenames()	Lista de rutas absolutas agregadas a archivos_seleccionados
2	Ordenar archivos con ▲ ▼ si es necesario	GestorLista.mover_arriba() GestorLista.mover_abajo()	Orden de procesamiento persistido en la misma lista
3	Clic "Iniciar"	app.iniciar_proceso_latencias()	Loop sobre archivos_seleccionados
4	(interno) Lectura de cada .mat	procesador_mat.leer_mat_a_df()	pd.DataFrame crudo (8 cols)
5	(interno) Limpieza de datos	procesador_mat.modificar_dataframe()	d.DataFrame limpio (5 cols, solo Desplazamiento > 1)
6	(interno) Acumulación	dfs_mat[nombre] = df_modificado	dict con N DataFrames (uno por rata)
7	Elegir ruta de guardado	filedialog.asksaveasfilename()	Ruta de salida del .xlsx
8	(interno) Escritura Excel	procesador_mat.guardar_excel()	.xlsx con hoja de promedios + hojas individuales por rata
9	Clic OK en messagebox	GestorLista.vaciar()	Lista de archivos limpiada; apps lista para nuevo lote
10	Cargar los .xlsx generados para graficar	app.seleccionar_archivos()	Misma lista con rutas de archivos Excel
11	Clic "Graficar"	app.procesar_xlsx() → graficador.generar_grafica()	Figura matplotlib + DataFrame de resumen estadístico

12	Revisar gráfica y guardar	fig.savefig() + guardar_excel_resumen()	Imagen .png (300 DPI) + Excel de resumen con tercios
----	---------------------------	---	--

12. Referencia completa de funciones

main.py - __main__	
Descripción	Punto de entrada. Crea la ventana CTK, aplica el tema ttk "clam" e inicia el event loop.
Parámetros	N/A
Retorno	N/A

app.py - __init__(self, root)	
Descripción	Inicializa toda la aplicación: configura la ventana, crea el fondo, construye la UI, instancia el gestor de lista, carga el logo e inicia la animación.
Parámetros	root → CTK()
Retorno	None

app.py - cargar_logo(self)	
Descripción	Busca el logo institucional en la ruta configurada. Si existe, lo redimensiona y lo muestra en el encabezado. Si no, muestra un texto alternativo.
Parámetros	None
Retorno	None

app.py - _limpiar_log(self)	
Descripción	Habilita temporalmente el CTKTextbox y elimina todo su contenido.
Parámetros	None
Retorno	None

app.py - <i>log(self, mensaje)</i>	
Descripción	Inserta una línea en el área de consola y aplica scroll automático al final.
Parámetros	mensaje: str
Retorno	None

app.py - <i>seleccionar_archivos(self)</i>	
Descripción	Abre el diálogo del SO para seleccionar archivos .mat y .xlsx. Pasa las rutas al gestor.
Parámetros	None
Retorno	None

app.py - <i>iniciar_proceso_latencias(self)</i>	
Descripción	Itera sobre la lista de archivos, lee cada .mat, limpia el DataFrame y exporta a Excel.
Parámetros	None
Retorno	None

app.py - <i>procesar_xlsx(self)</i>	
Descripción	Invoca generar_grafica(), muestra la figura matplotlib y guarda imagen PNG y Excel de resumen.
Parámetros	None
Retorno	None

interfaz.py - <i>construir_interfaz(app)</i>	
Descripción	Construye y empaqueta todos los widgets CTK de la ventana principal. Asigna referencias de widgets al objeto app.
Parámetros	app: caja_valentia_app
Retorno	None

interfaz.py - <i>_tarjeta(parent, texto_boton, texto_subtitulo, comando)</i>	
Descripción	Factory method que genera un bloque visual reutilizable con botón y subtítulo.
Parámetros	parent: CTKFrame, texto_boton: str, texto_subtitulo: str, comando: callable
Retorno	CTKFrame construido

lista_archivos.py - <i>actualizar_visual(self)</i>	
Descripción	Sincroniza el Listbox visual con la lista interna de archivos.
Parámetros	None
Retorno	None

lista_archivos.py - <i>anadir(self, nuevos_archivos)</i>	
Descripción	Añade nuevas rutas a la lista interna y actualiza la vista.
Parámetros	nuevos_archivos: tuple list
Retorno	None

lista_archivos.py - <i>mover_arriba(self)</i>	
Descripción	Intercambia el elemento seleccionado con el anterior
Parámetros	None
Retorno	None

lista_archivos.py - <i>eliminar(self)</i>	
Descripción	Elimina los elementos seleccionados en orden inverso para preservar índices.
Parámetros	None
Retorno	None

fondo_neuronal.py - <i>_construir_canvas(self)</i>	
Descripción	Crea el tk.Canvas oscuro con place relativo que cubre toda la ventana.
Parámetros	None
Retorno	None

fondo_neuronal.py - <i>_poblar_particulas(self)</i>	
Descripción	Genera los diccionarios de estado inicial para cada partícula del enjambre.
Parámetros	None
Retorno	None

fondo_neuronal.py - <i>animar(self)</i>	
Descripción	Motor gráfico recursivo: limpia el canvas, actualiza física, traza partículas y conexiones. Se auto-invoca cada 30 ms con after().
Parámetros	None
Retorno	None (llama self.root.after(30, self.animar))

procesador_mat.py - <i>leer_mat_a_df(ruta_archivo, log_fn)</i>	
Descripción	Lee un archivo .mat con scipy y lo convierte en DataFrame de pandas.
Parámetros	ruta_archivo: str, log_fn: callable
Retorno	pd.DataFrame None

procesador_mat.py - <i>modificar_dataframe(df)</i>	
Descripción	Elimina columnas innecesarias, filtra ensayos con Desplazamiento>1 y renumera.
Parámetros	df: pd.DataFrame
Retorno	pd.DataFrame limpio

procesador_mat.py - <i>calcular_promedios_latencia(dfs_dict, log_fn)</i>	
Descripción	Calcula promedio de latencia por condición (Estim Eléctrico 0/1) para cada archivo.
Parámetros	dfs_dict: dict[str, pd.DataFrame], log_fn: callable
Retorno	pd.DataFrame de promedios

procesador_mat.py - <i>guardar_excel(ruta_guardado, dfs_mat, log_fn)</i>	
Descripción	Escribe el Excel final con hoja de promedios+SEM y una hoja por rata con formato.
Parámetros	ruta_guardado: str, dfs_mat: dict, log_fn: callable
Retorno	bool (True si éxito)

13. Modelo de datos y estructuras

13.1 DataFrame Crudo (salida de leer_mat_a_df)

Columna	dtype pandas	Rango típico	Ejemplo
Ensayo	float64	[1, 100]	1.0, 2.0, 3.0
Lado	float64	{0, 1}	0.0, 1.0
Estim Electrico	float64	{0, 1}	0.0, 1.0
Latencia	float64	[0.1, 30.0]	4.73, 12.5
Tiempo Absoluto	float64	timestamp	1234.56
Palancas Izq	float64	[0, 50]	3.0
Palancas Der	float64	[0, 50]	1.0
Desplazamiento	float64	[0.0, 5.0]	2.13, 0.0
archivo_origen	object	str	"rata01_dia1.mat"

13.2 DataFrame Limpio (salida de modificar_dataframe)

Columna	dtype	Cambios respecto al crudo
Ensayo	float64	Renumerado secuencialmente (1,2,3,...) sin huecos
Lado	float64	Sin cambios
Estim Electrico	float64	Sin cambios
Latencia	float64	Sin cambios — valor analítico principal
Desplazamiento	float64	Sólo filas con Desplazamiento > 1

13.3 Ejemplo numérico completo del pipeline

Para ilustrar el flujo de transformación de datos, se muestra un ejemplo con datos simulados de un archivo de 6 ensayos, siguiendo cada etapa del procesamiento:

Etapas 1: Datos crudos de loadmat (6 ensayos)

Ensayo	Lado	Estim E.	Latencia	T.Abs.	P.Izq	P.Der	Desp.
1.0	0.0	0.0	3.45	100.2	2.0	0.0	0.0
2.0	1.0	1.0	8.72	108.9	0.0	3.0	2.1
3.0	0.0	0.0	4.11	113.0	1.0	0.0	0.0
4.0	1.0	1.0	15.3	128.3	0.0	2.0	1.8
5.0	0.0	0.0	2.98	131.2	1.0	0.0	0.0
6.0	1.0	0.0	6.50	137.7	0.0	1.0	3.2

Etapas 2: Aplicar `df[df["Desplazamiento"] > 1]` → filtra ensayos 1, 3, 5

Se eliminan los ensayos 1, 3 y 5 porque Desplazamiento=0.0 (el animal no se movió).

Ensayo orig.	Lado	Estim E.	Latencia	Despl.
2.0	1.0	1.0	8.72	2.1
4.0	1.0	1.0	15.3	1.8
6.0	1.0	0.0	6.50	3.2

Etapas 3: `reset_index()` + reenumerar Ensayo

Ensayo	Lado	Estim E.	Latencia	Despl.
1	1.0	1.0	8.72	2.1
2	1.0	1.0	15.3	1.8
3	1.0	0.0	6.50	3.2

14. Manejo de errores y robustez

El sistema implementa un enfoque de fallo suave: los errores en archivos individuales son capturados y reportados en la consola sin detener el procesamiento del resto del lote. El usuario siempre recibe retroalimentación.

Escenario de error	Mecanismo de detección	Respuesta del sistema
Lista de archivos vacía al procesar	if not self.archivos_seleccionados	messagebox.showwarning() + return
Archivo .mat corrupto o ilegible	try/except en leer_mat_a_df	log(f"Error al procesar...") + return None

Número de columnas incorrecto	<code>matriz_datos.shape[1] != 8</code>	<code>log("Error en dimensiones...") + return None</code>
Variable de datos no encontrada en .mat if	<code>matriz_datos is None</code>	<code>log("No se encontraron datos...") + return None</code>
Error al escribir el archivo Excel	<code>try/except</code> en <code>guardar_excel</code>	<code>log("Error crítico...") + return False + messagebox.showerror()</code>
Logo no encontrado	<code>if os.path.exists(ruta_imagen)</code>	Texto alternativo [Valentía Lab] en el encabezado
Sin ensayos válidos tras el filtro	DataFrame vacío (<code>len==0</code>)	Se guarda igual, la hoja queda vacía (no crashea)
Columnas requeridas ausentes en promedios	<code>"Latencia" not in df.columns</code>	<code>log(f"Advertencia: {nombre} no tiene las columnas requeridas")</code>

14.1 Patrón de log_fn Inyectado

Todas las funciones de la capa lógica reciben un parámetro `log_fn=print` como función de reporte de estado. Esto desacopla completamente la lógica de negocio de la interfaz gráfica: durante pruebas unitarias se puede usar `print`, y en producción se usa `app.log`.

15. Dependencias, Entorno e Instalación

15.1 Requisitos del Sistema

- Python 3.8 o superior (recomendado 3.10+)
- Sistema operativo: Windows 10/11 (desarrollo original), compatible con Linux/macOS con ajustes de ruta)
- Resolución mínima de pantalla: 1280 × 800 px

15.2 Instalación de dependencias

```
# Instalar todas las dependencias con pip pip install customtkinter scipy pandas openpyxl matplotlib Pillow
# requirements.txt equivalente customtkinter>=5.2.0 scipy>=1.9.0 pandas>=1.5.0 openpyxl>=3.0.10
matplotlib>=3.6.0 Pillow>=9.0.0
```

15.3 Ejecución

Desde la raíz del proyecto ***cd caja_valentia/ python main.py***

16. Conclusiones y recomendaciones

16.1 Logros técnicos

- Automatización completa del pipeline de análisis: desde archivo .mat crudo hasta reporte Excel con estadísticas.
- Separación clara de responsabilidades siguiendo principios SOLID y el patrón MVC simplificado.
- Interfaz gráfica completamente responsiva que se adapta al tamaño de la ventana en tiempo real.
- Manejo robusto de errores con retroalimentación visual inmediata sin interrumpir el procesamiento del lote.
- Sistema de animación eficiente para 60 partículas a ~33 FPS sin degradar la UI.
- Código modular y desacoplado que facilita el mantenimiento y la incorporación de nuevas funciones.

16.2 Métricas del código

Métrica	Valor
Módulos Python	7 archivos
Clases definidas	2 (FondoNeuronal, GestorLista)
Funciones / métodos	~25
Líneas de código (aprox.)	~650
Bibliotecas externas	6
Diálogos modales del SO	4 (2 abrir, 2 guardar)
Formatos de archivo soportados como entrada	2 (.mat, .xlsx)
Formatos de archivo generados como salida	2 (.xlsx, .png)

Apéndice A - Referencia Rápida de y Controles

Guía rápida de uso para el personal de laboratorio sin conocimientos técnicos:

Acción	Control
Cerrar la aplicación	Tecla ESC
Seleccionar archivos .mat para procesar Botón "Seleccionar Archivos" → sección 1	Seleccionar archivos .mat para procesar Botón "Seleccionar Archivos" → sección 1
Seleccionar archivos .xlsx para graficar	Botón "Seleccionar para Gráfica" → sección 1
Subir un archivo en la cola	Seleccionar en lista → botón ▲ Subir
Bajar un archivo en la cola	Seleccionar en lista → botón ▼ Bajar
Eliminar un archivo de la cola	Seleccionar en lista → botón ✕ Eliminar
Selección múltiple en la lista	Ctrl+Clic (Windows) para seleccionar varios
Procesar archivos .mat a Excel	Botón "Iniciar" → sección 2
Generar gráfica comparativa	Botón "Graficar" → sección 2
Ver detalles del proceso	Consola verde en la parte inferior de la ventana

Apéndice B - Diagnóstico de Problemas Frecuentes

Síntoma / Error	Causa probable	Solución
"Error en dimensiones: X cols"	El archivo .mat no tiene 8 columnas; pertenecía a otro experimento.	Verificar que el archivo sea del experimento correcto de la Caja de Valentía
"No se encontraron datos válidos"	El archivo .mat está vacío, corrupto o las claves no son reconocidas por scipy.	Re-exportar desde MATLAB. Verificar con <code>scipy.io.loadmat()</code> manualmente.
La lista de archivos está vacía después de cargar	Los archivos no tienen extensión .mat o .xlsx permitida.	Renombrar los archivos con la extensión correcta antes de cargar

La ventana parece congelada durante el procesamiento	El lote es muy grande y el procesamiento tarda en completarse.	Esperar. El log se actualiza cuando cada archivo termina.
El logo no aparece en el encabezado	La ruta <code>FILENAME_LOGO</code> no existe en el equipo donde se ejecuta.	Actualizar la constante en <code>app.py</code> con la ruta correcta del archivo <code>.png</code>
Error al instalar dependencias	Python no está en el <code>PATH</code> del sistema o <code>pip</code> no está disponible.	Instalar Python 3.10+ desde <code>python.org</code> y marcar "Add to <code>PATH</code> " durante la instalación.
La gráfica no se muestra	<code>matplotlib</code> backend incompatible con el entorno de ejecución.	Agregar <code>import matplotlib; matplotlib.use("TkAgg")</code>